VIT

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

LA 5

Name – Garv Rawat

Registration Number - 23BLC1058

Faculty Name - Dr. Brindha S

Course - Software Engineering Lab (BCSE301P)

School of Electronics Engineering (SENSE)

Package Diagram

1. Overview of the Package Diagram

The **Stock Trading & Risk Management System Package Diagram** represents the high-level organization of the system by grouping related components into logical packages. It illustrates how different layers such as presentation, API handling, trading logic, risk management, and external integrations are structured and interact with each other.

The system follows a **layered architecture**, ensuring clear separation of concerns, improving scalability, maintainability, and performance. Each package encapsulates specific responsibilities, allowing independent development and efficient system management.

2. Package Diagram Description

2.1 Presentation Layer

Components:

- Trader Interface
- Admin Dashboard

Responsibilities:

- Captures user inputs (buy/sell orders, portfolio view)
- Displays stock prices, reports, and analytics
- Communicates with API Layer via REST

2.2 API Layer

Components:

- API Gateway

Responsibilities:

- Handles incoming REST requests
- Routes requests to trading and portfolio services
- Ensures standardized communication
- Interacts with Security Layer

2.3 Security Layer

Components:

- Authentication
- Authorization

Responsibilities:

- Verifies user identity
- Controls access to trading features
- Ensures secure transactions

2.4 Business Logic Layer

Components:

- User Management
- Trade Execution Engine
- Portfolio Management
- Risk Analysis Engine
- Reporting Service
- Market Data Service

Responsibilities:

- Processes trade requests
- Executes buy/sell operations
- Tracks portfolio performance
- Performs risk analysis (limits, exposure)
- Generates reports and analytics
- Fetches real-time stock data

2.5 External Systems Layer

Components:

- Stock Exchange API
- Payment Gateway
- Notification Service

Responsibilities:

- Executes real market trades
- Handles payments and settlements
- Sends alerts and notifications

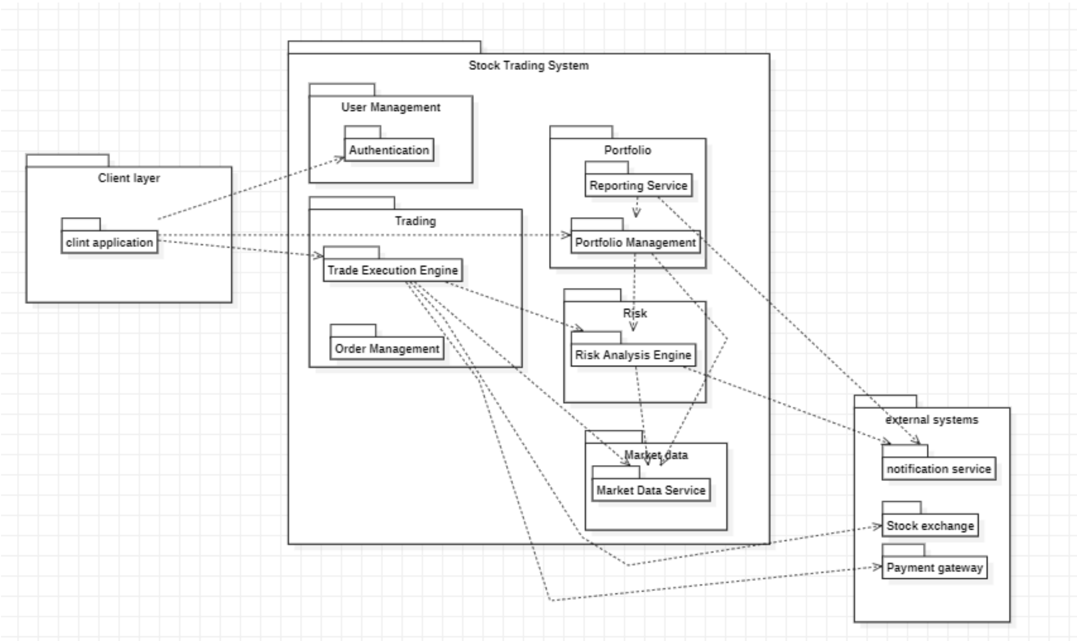
3. Interaction Between Packages

1. The Presentation Layer sends requests to the API Layer.
2. The API Gateway routes requests to appropriate services.
3. The Security Layer validates authentication and authorization.
4. The Business Logic Layer:
    - Executes trades
    - Performs risk checks
    - Updates portfolio

5. External Systems:
    - Execute trades (Stock Exchange)
    - Handle payments
6. Responses are returned to the user via API Layer.

4. Key Design Features

- **Layered Architecture** → clear separation of UI, logic, APIs
- **Modularity** → independent services
- **Scalability** → trading and risk modules scale independently
- **Security** → protected transactions
- **Maintainability** → easy updates



5. Conclusion

The package diagram provides a structured view of the stock trading system using layered architecture. It ensures scalability, security, and modular design, making it suitable for real-world financial systems.

Component Diagram

1. Overview of the Component Diagram

The **Stock Trading System Component Diagram** illustrates how different components interact to perform trading operations, risk evaluation, and portfolio management. It highlights modular design and service interaction.

2. Component Diagram Description

2.1 Major Components

- **User Interface**
  - Entry point for traders
- **REST API / API Gateway**
  - Handles communication
  - Routes requests

2.2 Core Backend Services

- **Auth Service**
  - User authentication
- **Trade Execution Engine**
  - Executes buy/sell orders
- **Portfolio Service**
  - Tracks investments
- **Risk Analysis Engine**
  - Checks risk before execution
- **Market Data Service**
  - Fetches real-time stock prices
- **Reporting Service**
  - Generates analytics

2.3 External Components

- **Stock Exchange API**
- **Payment Gateway**
- **Notification Service**

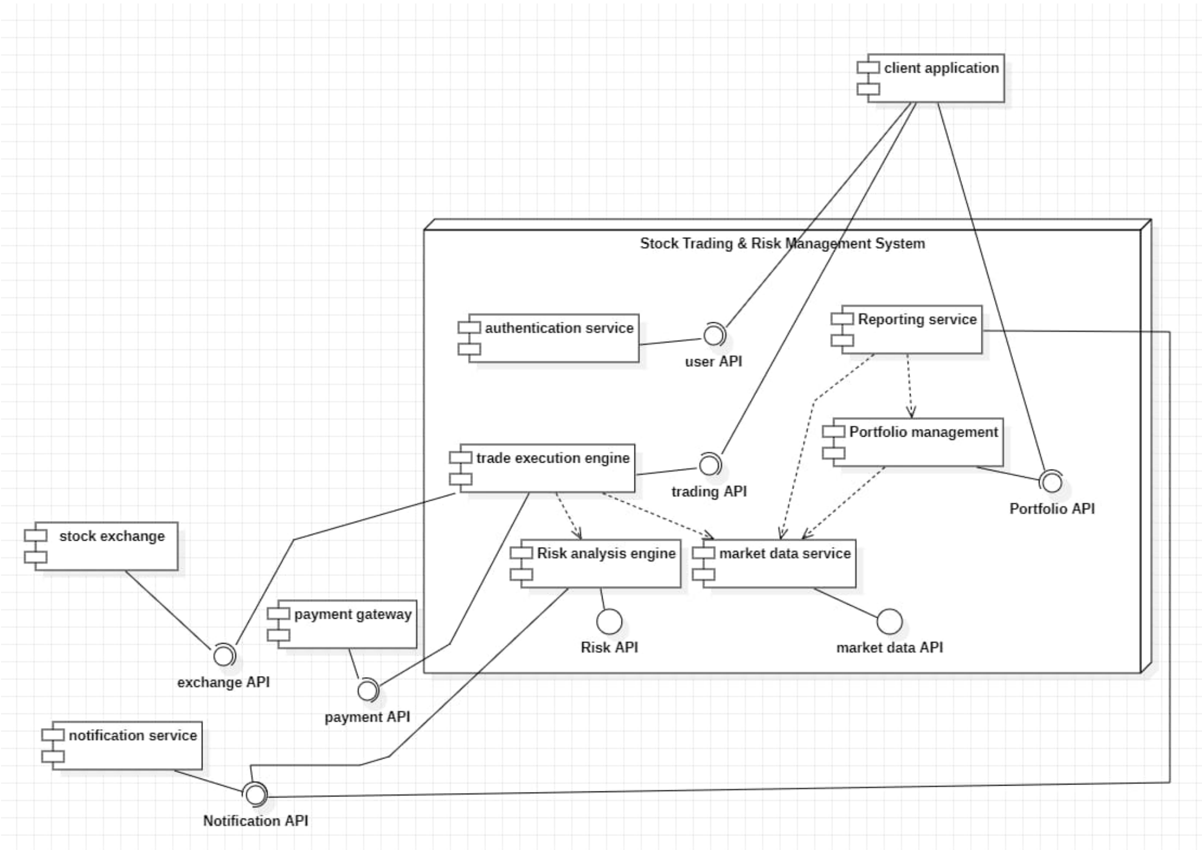
3. Interaction Flow

1. User sends request via UI
2. API Gateway processes request
3. Auth Service validates user
4. Trade Engine:
   - Fetches market data
   - Checks risk

- Executes trade
5. Portfolio updates
6. Reports generated
7. Notifications sent

4. Key Design Features

- Modular services
- Scalable architecture
- Secure transactions
- Real-time processing



5. Conclusion

The component diagram shows a scalable and modular trading system with clear service responsibilities, enabling efficient trade execution and risk management.

Deployment Diagram

1. Overview of the Deployment Diagram

The **Deployment Diagram** represents how the stock trading system is deployed across different hardware nodes such as client devices, servers, and external systems.

2. Deployment Diagram Description

2.1 Client Layer

Node: Client Device

Artifacts:

- Web App
- Mobile App

Responsibilities:

- Sends requests
- Displays trading data

2.2 Web Server

Artifacts:

- API Gateway
- Authentication Service

Responsibilities:

- Handles requests
- Performs validation

2.3 Application Server

Artifacts:

- Trade Engine
- Portfolio Service
- Risk Engine
- Market Data Service
- Reporting Service

Responsibilities:

- Executes business logic
- Processes trades and analytics

2.4 External Systems

Artifacts:

- Stock Exchange API
- Payment Gateway
- Notification Service

Responsibilities:

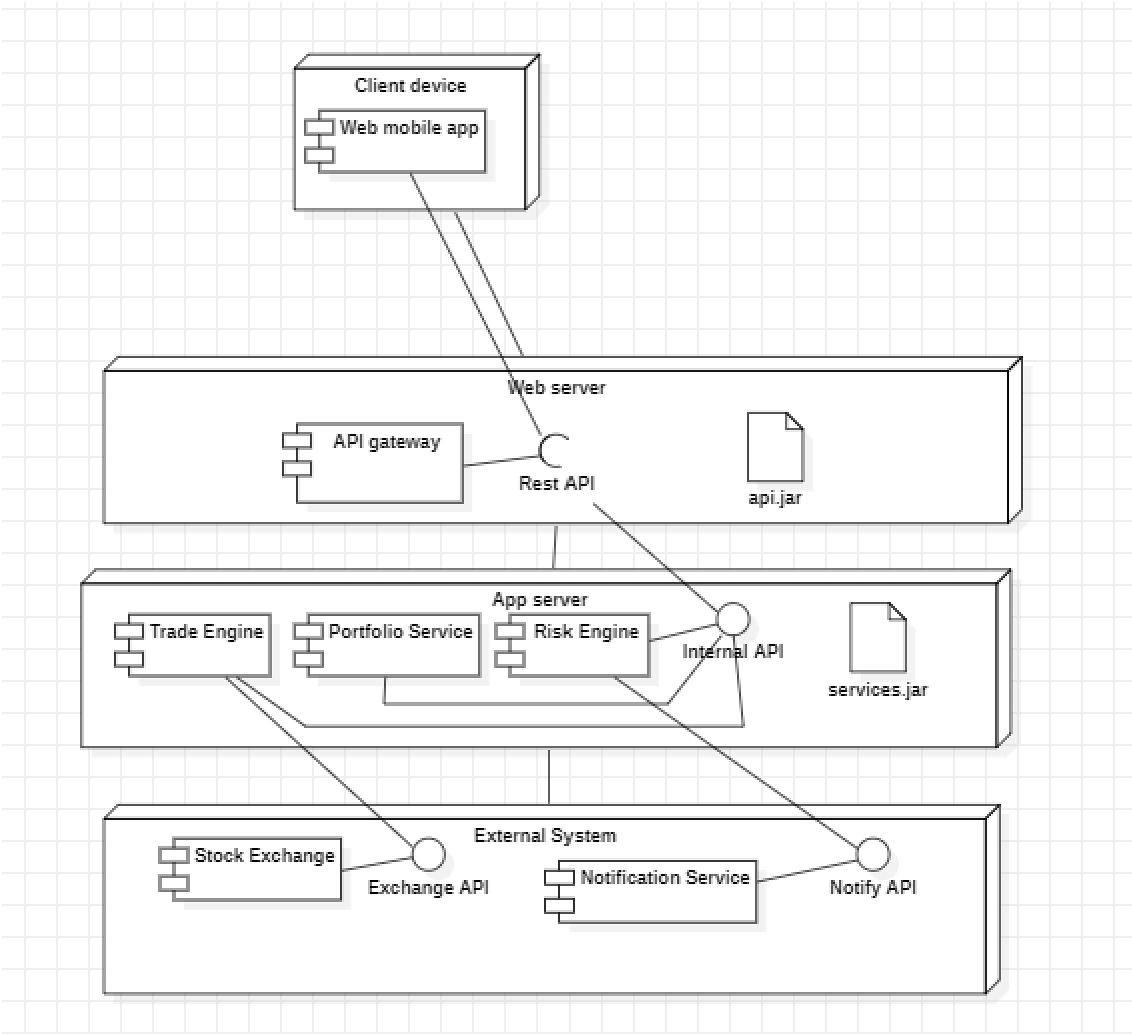
- Trade execution
- Payment processing
- Alerts

3. Interaction Flow

1. Client sends request
2. Web Server processes request
3. Application Server executes logic
4. External systems handle trades/payments
5. Response returned to client

4. Key Design Features

- Distributed architecture
- High scalability
- Real-time processing
- Secure communication



5. Conclusion

The deployment diagram demonstrates a robust infrastructure supporting real-time trading, ensuring performance, scalability, and reliability.

TestCase Studio

Overview:
To test the login functionality and file management features (deleting a file) of Google Drive using different input conditions and record test cases using TestCase Studio.

Google Drive is a cloud storage service provided by Google that allows users to store, access, and manage files online. It requires authentication through a Google account.

Testing login functionality ensures:

- Proper validation of email/username
- Detection of incorrect password attempts
- Secure authentication process
- Successful navigation to the dashboard after login

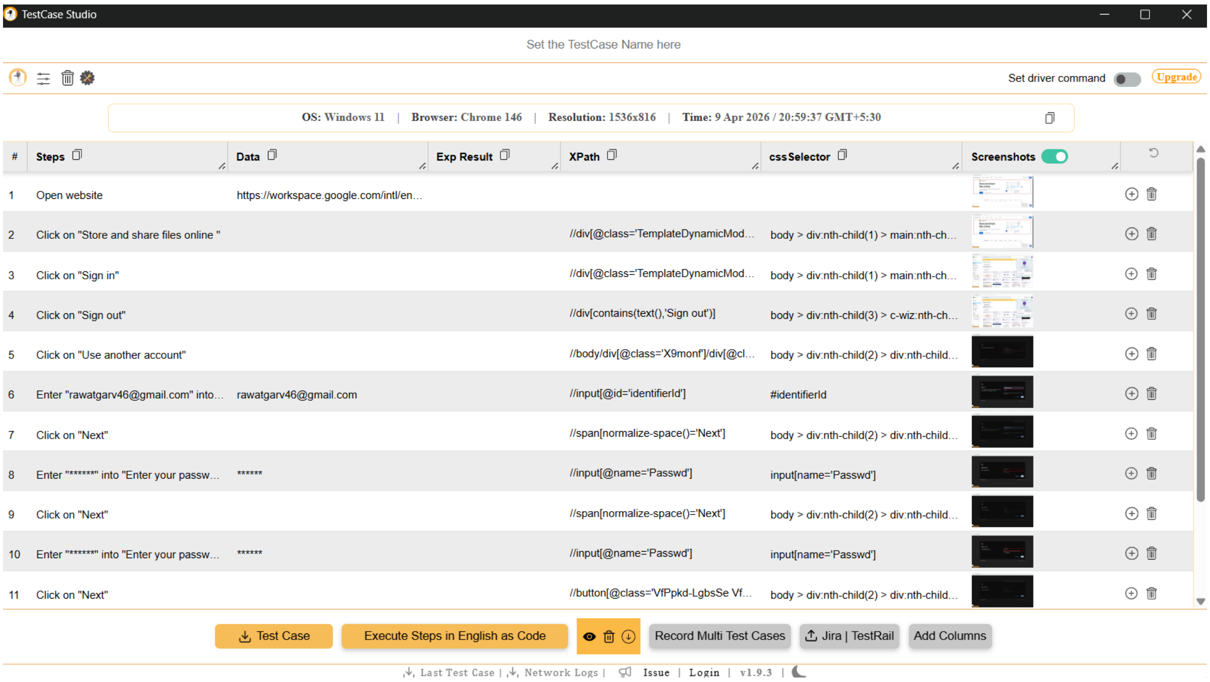
Additionally, testing file management functionality ensures:

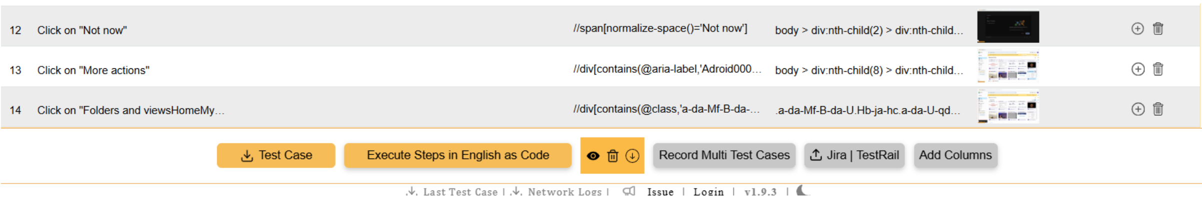
- Users can perform operations like delete, upload, and manage files
- Proper system response after deleting a file
- File is moved to Trash instead of being permanently deleted immediately
- User interface responds correctly to file actions

TestCase Studio helps in capturing all user actions and generating structured test cases automatically.

Testing includes:

- Negative Testing → Valid email with incorrect password
- Positive Testing → Correct login credentials
- Functional Testing → File deletion operation



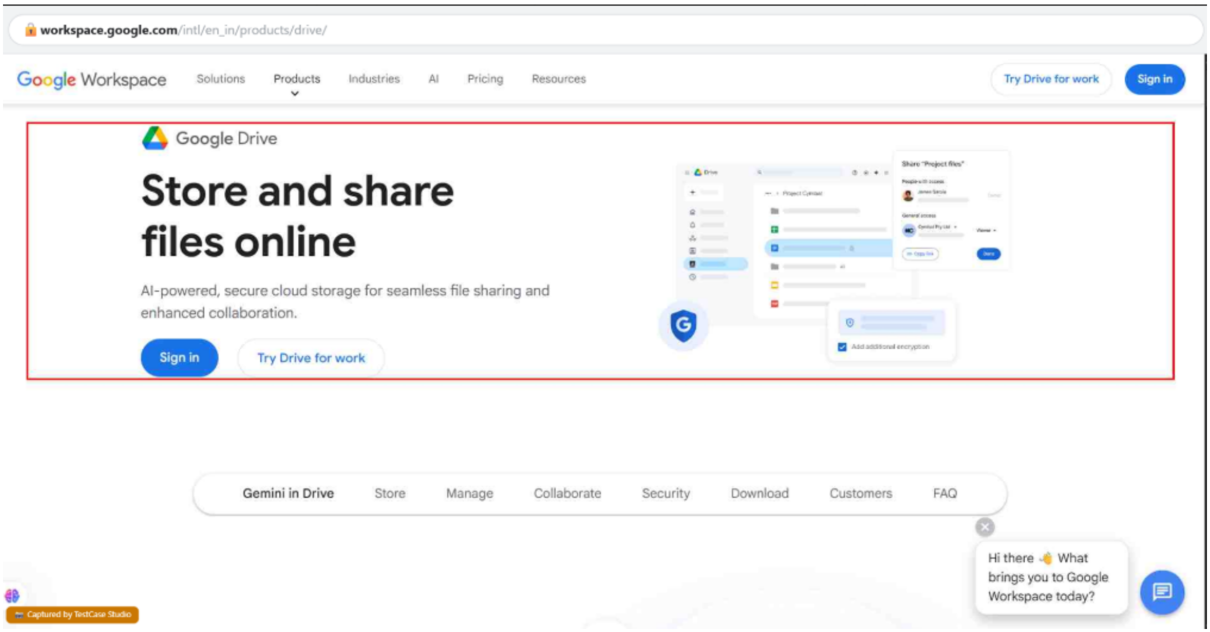


Test Steps

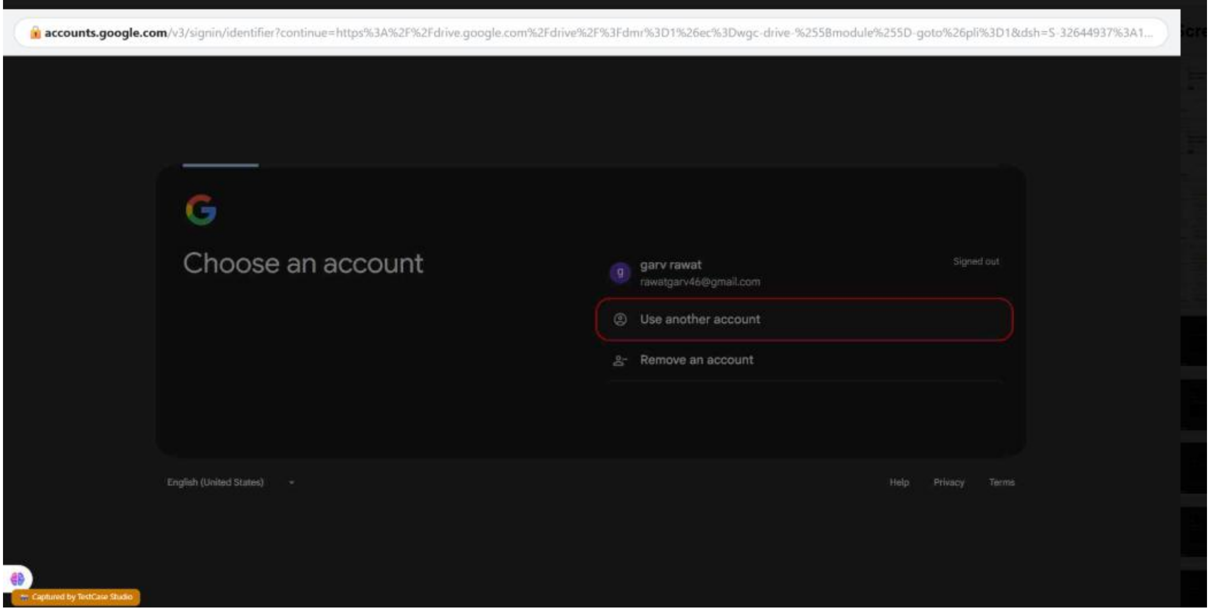
Test Case 1: Valid Email with Invalid Password

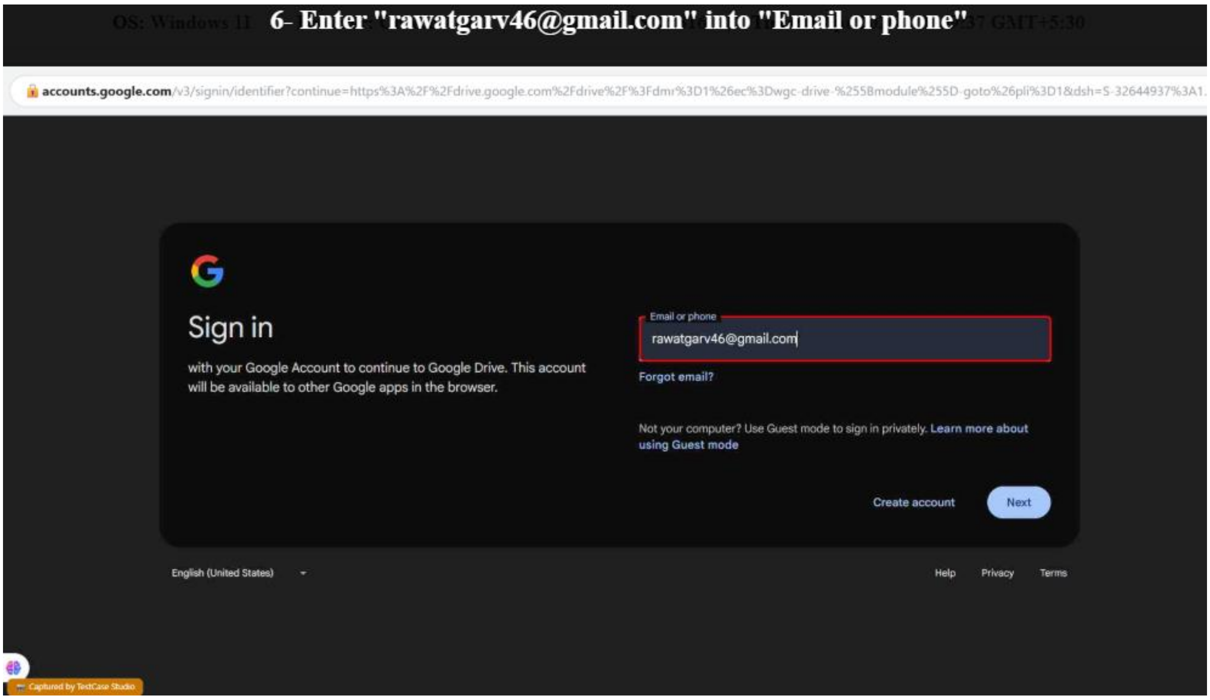
1. Open the Google Drive login page.
2. Enter a valid email address.
3. Click on Next.
4. Enter an incorrect password.
5. Click on Sign in.

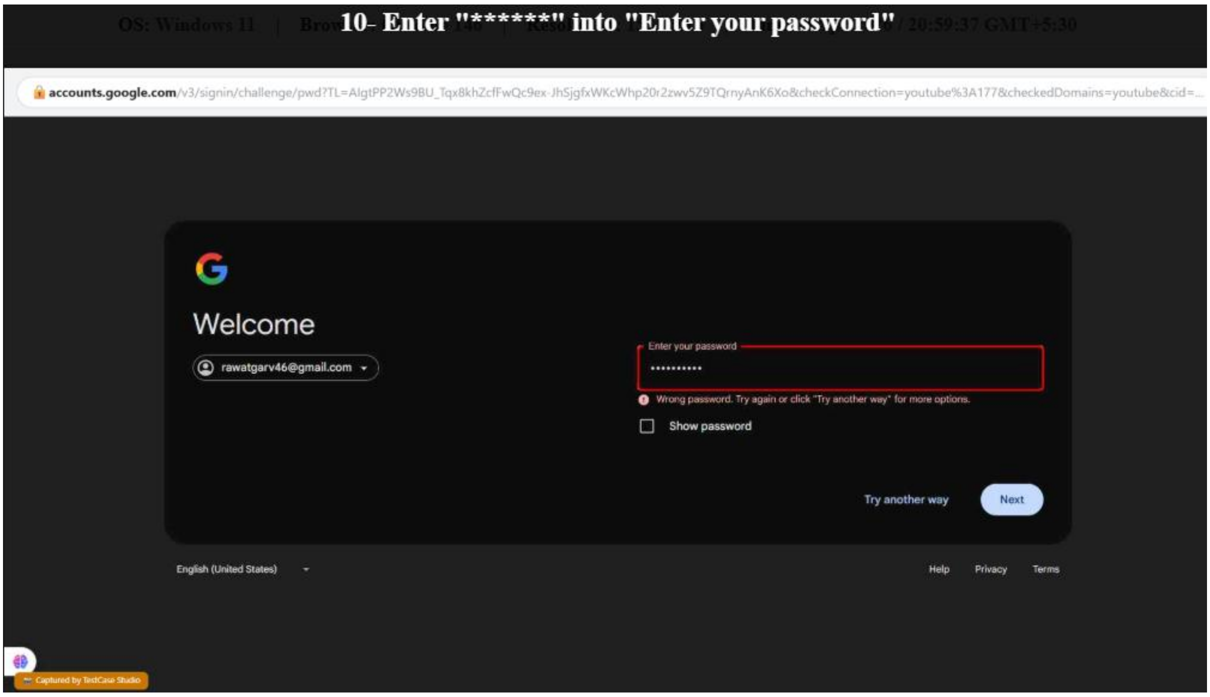
Expected Result:
The system should display an error message indicating that the password is incorrect.



5- Click on "Use another account"





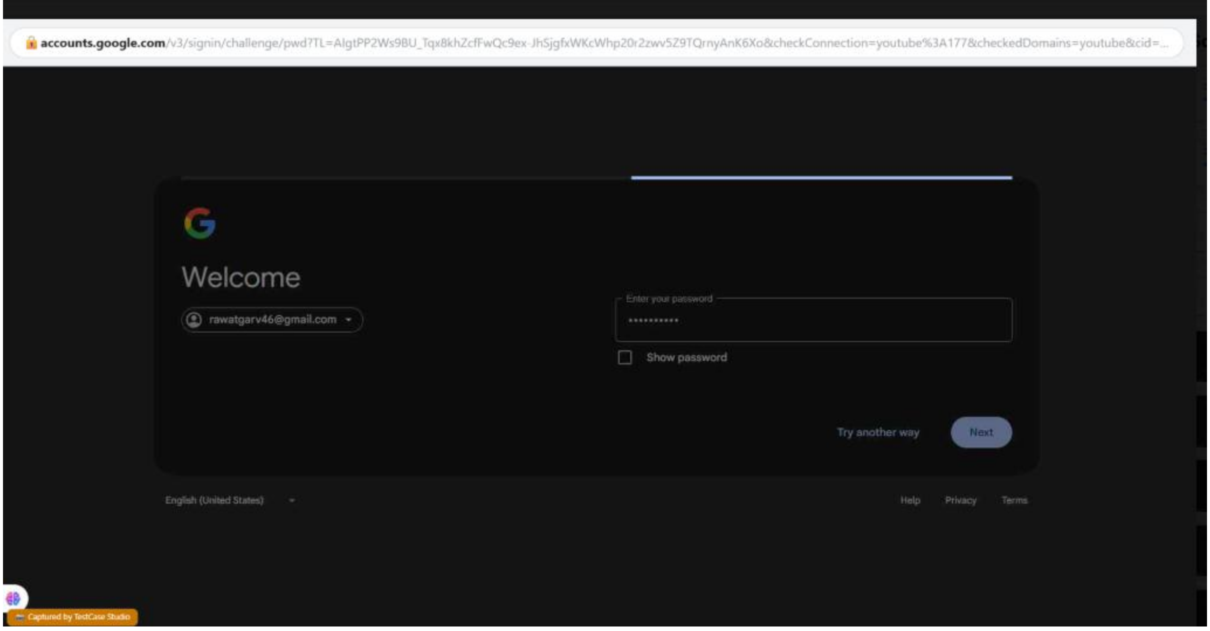


Test Case 2: Successful Login

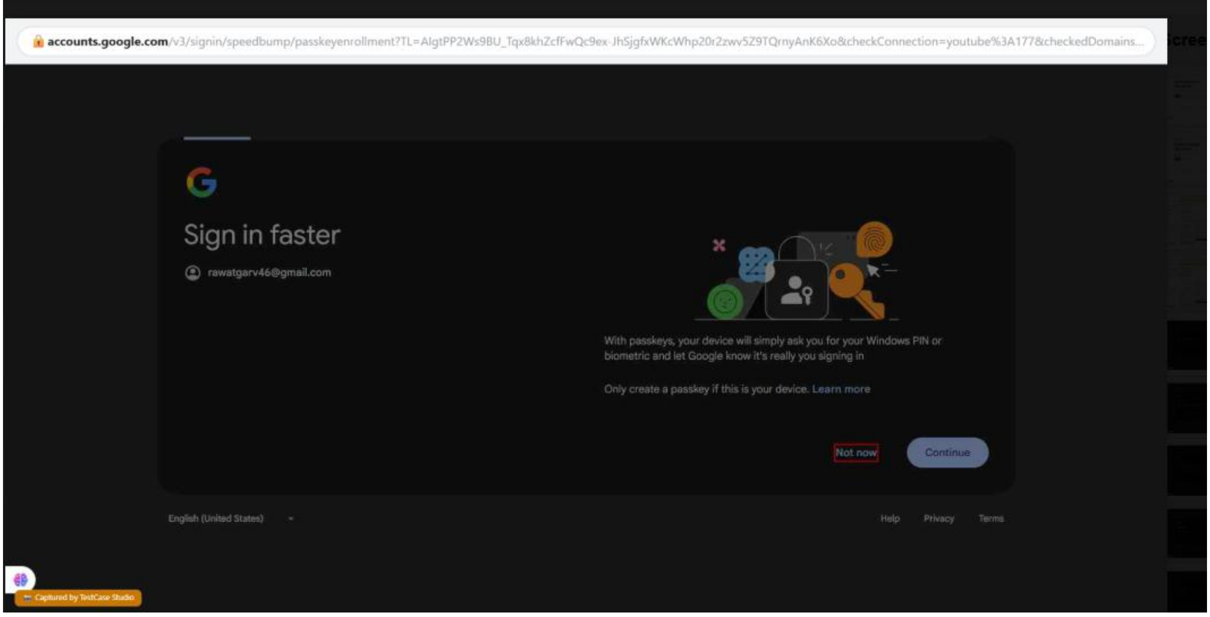
6. Enter the correct password.
7. Click on Sign in.
8. Complete verification if prompted.

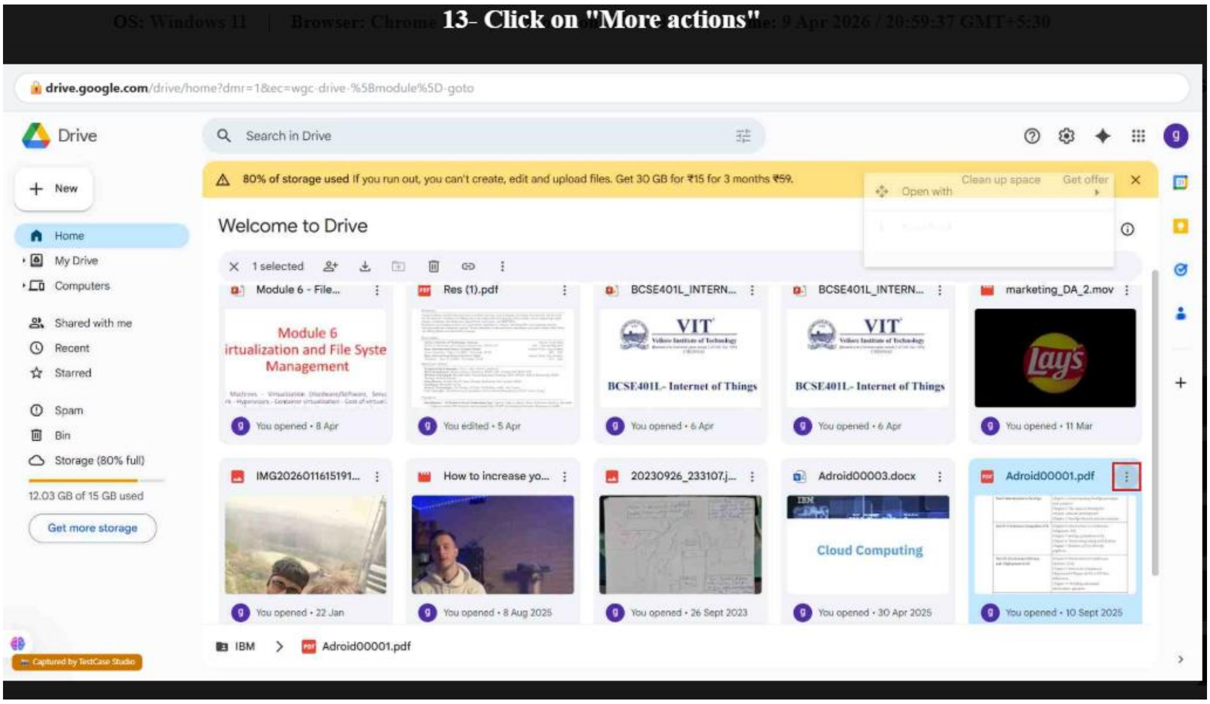
Expected Result:
The user should be successfully logged into Google Drive dashboard.

11- Click on "Next"



12- Click on "Not now"

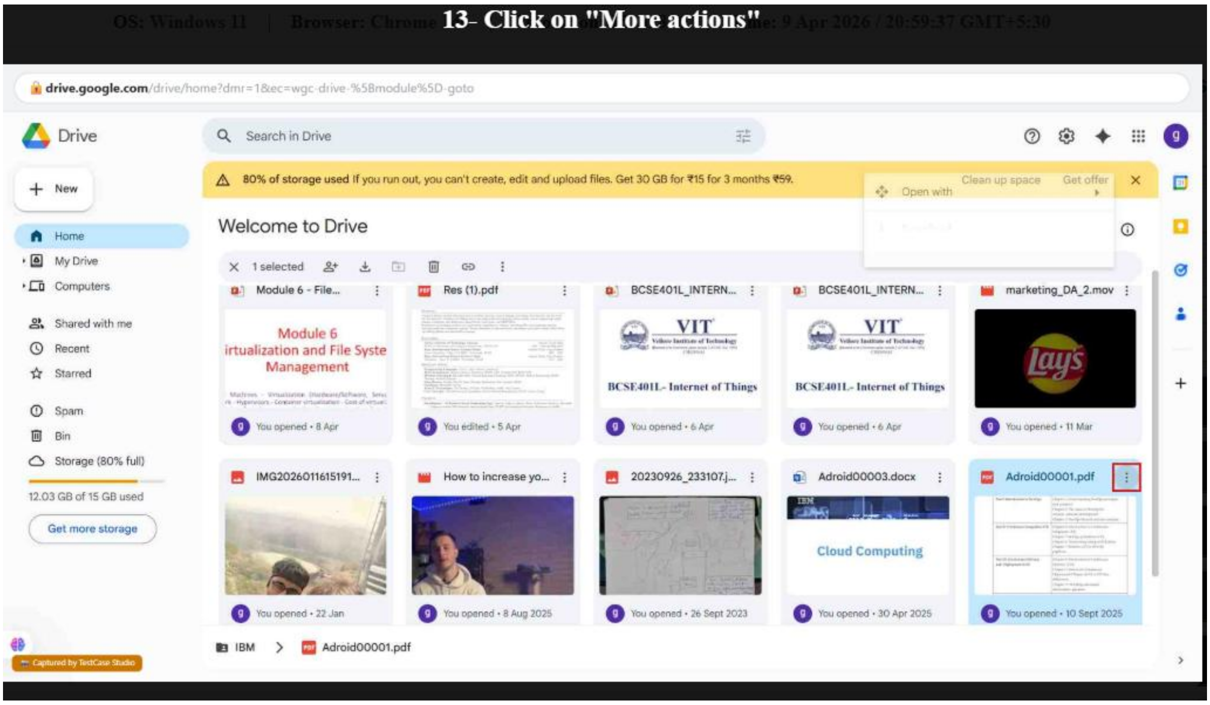


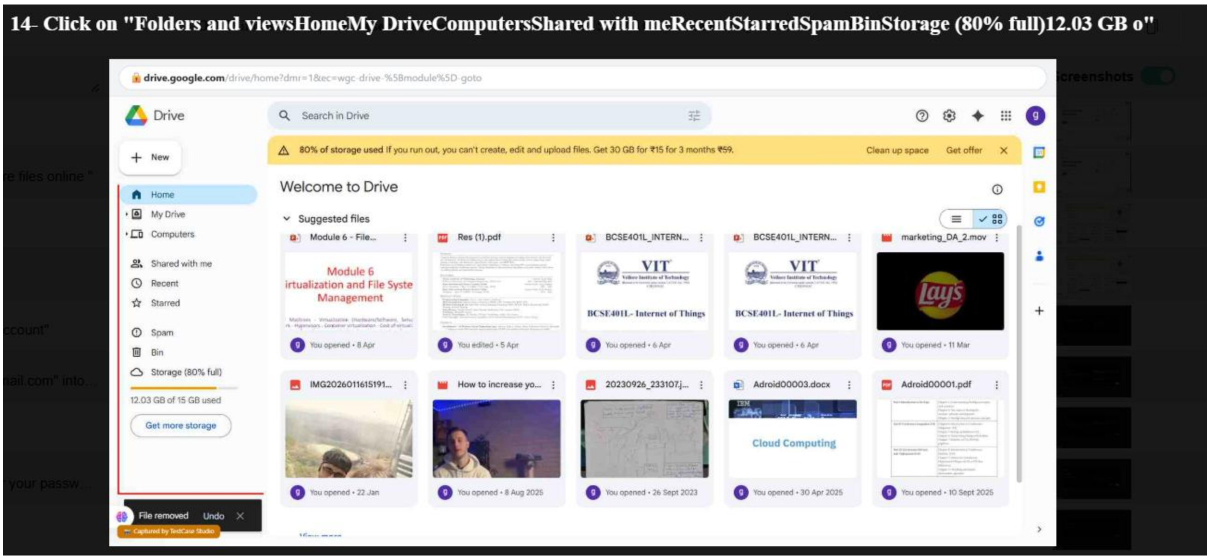


Test Case 3: Delete File Functionality

9. Select a file from Google Drive.
10. Right-click on the file or click on the delete icon.
11. Click on “Remove” (Delete).

Expected Result:
The selected file should be removed from My Drive and moved to the Trash folder.





Excel Report:

#	Step	Data	Exp Resu	XPath	cssSelector
	OS: Windows 11 | Browser: Chrome 146 | Resolution: 1536x816 | Time: 9 Apr 2026 / 21:3:39 GMT+5:30				
1	Open we	https://workspace.google.com/intl/en_in/products/drive/			
2	Click on "Store and share fil			//div[@cl	body > div:nth-child(1) > main:nth-child(2) > section:nth-child(2) > gws-dynamic-module-container:nth-child(1) > div:nth-child(2) > section:nth-child(1) > div:nth-child(1) > div:nth-child(1)
3	Click on "Sign in"			//div[@cl	body > div:nth-child(1) > main:nth-child(2) > section:nth-child(2) > gws-dynamic-module-container:nth-child(1) > div:nth-child(2) > section:nth-child(1) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > div:nth-child(3) > div:nth-child(2) > span:nth-child(1) > a:nth-child(1) > span:nth-child(1)
4	Click on "Sign out"			//div[com	body > div:nth-child(3) > c-wiz:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > div:nth-child(2) > div:nth-child(2) > div:nth-child(1) > div:nth-child(3) > div:nth-child(3) > span:nth-child(1) > span:nth-child(2) > a:nth-child(1) > div:nth-child(1) > span:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1)
5	Click on "Use another accou			//body/di	body > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(2) > c-wiz:nth-child(1) > main:nth-child(1) > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > span:nth-child(1) > section:nth-child(1) > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > ul:nth-child(1) > li:nth-child(2) > div:nth-child(1)
6	Enter "ra	rawatgarv46@gma		//input[@	#identifierId
7	Click on "Next"			//span[nc	body > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(2) > c-wiz:nth-child(12) > main:nth-child(1) > div:nth-child(3) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > button:nth-child(1) > span:nth-child(4)
8	Enter "** ******			//input[@	input[name='Passwd']
9	Click on "Next"			//span[nc	body > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(2) > c-wiz:nth-child(12) > main:nth-child(1) > div:nth-child(3) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > button:nth-child(1) > span:nth-child(4)
10	Enter "** ******			//input[@	input[name='Passwd']
11	Click on "Next"			//button[(	body > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(2) > c-wiz:nth-child(12) > main:nth-child(1) > div:nth-child(3) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > div:nth-child(1) > button:nth-child(1) > div:nth-child(3)
12	Click on "Not now"			//span[nc	body > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > div:nth-child(2) > c-wiz:nth-child(1) > main:nth-child(1) > div:nth-child(3) > div:nth-child(1) > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > button:nth-child(1) > span:nth-child(4)
13	Click on "More actions"			//div[com	body > div:nth-child(8) > div:nth-child(1) > div:nth-child(6) > div:nth-child(2) > div:nth-child(1) > div:nth-child(1) > c-wiz:nth-child(1) > div:nth-child(1) > div:nth-child(1) > div:nth-child(2) > c-wiz:nth-child(4) > div:nth-child(1) > div:nth-child(1) > section:nth-child(3) > div:nth-child(2) > div:nth-child(1) > div:nth-child(10) > div:nth-child(1) > div:nth-child(2) > div
14	Click on "Folders and viewsl			//div[com	.a-da-Mf-B-da-U.Hb-ja-hc.a-da-U-qdBTIf-Zf

Conclusion:

Using TestCase Studio, both login and file deletion functionalities of Google Drive were successfully tested. The system correctly handled invalid login attempts, allowed access with valid credentials, and successfully moved deleted files to Trash, confirming proper functionality.